

altairsim — CP/M 2.2b on an Altair 8" floppy

2026-09-25 · 7959869

To launch it, name the machine file from inside this folder:

```
$ altairsim cpm22-buffered.toml
```

On **Windows** the program is `altairsim.exe`:

```
> altairsim.exe cpm22-buffered.toml
```

The machine boots itself — you type no `B00T` command — and comes up at the CP/M prompt. Type `DIR` to see the disk:

```
56K CP/M 2.2b v2.3
For Altair 8" Floppy

A>DIR
```

Mike Douglas's track-buffered **CP/M 2.2b v2.3**, on an 8" Pertec FD-400 behind an 88-DCDD, booted by the DBL PROM at `FF00` — `RUN FF00` is the machine file's whole startup, because on a real disk Altair that was `EXAMINE FF00` and `RUN`.

`^E` (STOP) takes the keyboard back to the monitor at any point; `RUN` resumes. `^C` belongs to CP/M (it is warm boot) and CP/M gets it.

The files

File	What it is
<code>cpm22-buffered.toml</code>	The machine: <code>base = "default"</code> plus the floppy in drive 0. Read it — it explains the memory arithmetic and what <code>readonly</code> really does on this controller.
<code>cpm22-terminal.toml</code>	The same machine and the same disk, but the console is a built-in VT100 window the simulator draws itself instead of <code>stdio</code> . The monitor stays in the terminal you launched from; CP/M comes up in its own window. No telnet client, no external emulator. Try <code>emulation=adm3a</code> for a period CP/M terminal.
<code>cpm22-fdcplus.toml</code>	CP/M from an FDC+ Serial Drive Server : no disk in this folder is used. The FDC+ gets its tracks from the server over a serial cable. Set your serial port in the file first — see below.
<code>cpm22b23-56k.dsk</code>	The bootable system disk, built for a 56K machine. Carries <code>DDT.COM</code> , <code>M80 / L80</code> , <code>MBASIC</code> and the host-bridge utilities (<code>R</code> , <code>W</code> , <code>HDIR</code>), with 18K free. Shared by both machine files above.

There is no undo. Drive 0 is mounted read/write because that is what a real machine is, and CP/M writes to `A:` for anything you create. In a clone `git checkout` puts the image back; in the package you were handed, nothing does. Copy it first if you are about to test writes in anger.

Three drives are empty. `MOUNT dsk0:drive1 "my-scratch.dsk"` fills one, or `FORMAT` from inside CP/M will make you a disk.

`BOOT.ASM` and `BIOS.ASM` — this CP/M's own, and the authoritative source the 88-DCDD was built from — are in `disks/mits-88dcdd/cpm22/buffered/`. They are source rather than product, so they stay in the repository and are not in the package.

CP/M from an FDC+ drive server

`cpm22-fdcplus.toml` is the same Altair with an **FDC+** in place of the 88-DCDD, in its serial-drive mode. The disk images are on an **FDC+ Serial Drive Server** (or any program that speaks its protocol) at the other end of a serial cable, and the board gets them a track at a time. The simulator and a real FDC+ Altair can use the same images through the same server.

1. Start the server, and mount a bootable 8" CP/M image in its drive 0. `cpm22b23-56k.dsk` from this folder will do.
2. Open `cpm22-fdcplus.toml` and set `connect` on `fdc0` to your serial port. The file has examples for macOS (`serial:/dev/cu.usbserial-XXXX`), Linux (`serial:/dev/ttyUSB0`) and Windows (`serial:COM3`).
3. Set `baud` to the rate that the server uses: 9600, 19200, 38400, 57600, 76800, 230400, 403200 or 460800. The two ends must agree.

4. Run it:

```
$ altairsim cpm22-fdcplus.toml
```

If the port does not open, the error lists the serial ports this computer has.

The file sets `clock_hz = 10000000` and `baud = 230400`. The crystal must not be full speed: the CP/M BIOS gives up on a sector after a count that takes 0.28 seconds at 10 MHz, and a track takes 0.19 seconds at 230400 baud. At full speed the count takes a few milliseconds and no track arrives in time. On a slower line, slow the crystal down too: at 38400 baud a track takes about one second, which needs the real 2 MHz.